□

# Woofer Care AI

## Complete Code Explanation Guide

*For startup pitches, technical interviews, and presentations*

**□ What this document covers**

- Full explanation of every section of woofer_care_ai.py
- What each technology is and why it was chosen
- How to explain the architecture to non-technical audiences
- Key talking points for startup meetings and investor pitches
- Answers to common technical questions you may be asked

## 1. The Big Picture — What Does This App Do?

Woofer Care AI is a single Python file (~1,100 lines) that runs as a full web application. When someone opens the app in a browser, they can upload a photo of their dog, and the app automatically:

- Identifies the dog breed using an AI image recognition model
- Looks up breed-specific care information from a built-in knowledge base
- Shows personalized recommendations for nutrition, exercise, grooming, and health
- Generates a downloadable PDF care guide
- Lets users track health appointments and reminders
- Provides links to buy supplies from Azerbaijani online stores

All of this happens in one file, deployed live on Streamlit Cloud, accessible from any device with a browser — no installation needed for the end user.

## 2. The Imports — What Libraries Are Used?

The first ~15 lines import all the external tools the app needs. Think of imports like hiring specialists — each one brings a specific skill to the project.

| Import | What it does & why we use it |
|---|---|
| streamlit | The entire web app framework. Turns Python code into a browser UI with buttons, file uploaders, tabs, charts — no HTML needed. This is why the app has a visual interface. |
| tensorflow | Google's AI/machine learning library. We use it to run the neural network that identifies dog breeds from photos. |
| MobileNetV2 | A specific pre-trained AI model inside TensorFlow/Keras. Trained on 1,000 image categories including 120+ dog breeds. We use it 'as-is' without retraining — this is called transfer learning. |
| preprocess_input | A helper function that formats the image correctly before feeding it into MobileNetV2 (resizes, normalizes pixel values to what the model expects). |
| decode_predictions | Translates the model's raw number output (1,000 probabilities) into human-readable labels like 'golden_retriever' with a confidence percentage. |
| keras_image | Handles loading and converting images into numerical arrays that the AI model can process. |
| numpy (np) | The foundation of numerical computing in Python. Used to manipulate the image as a matrix of numbers before AI analysis. |
| PIL / Pillow | Python Imaging Library. Opens and processes uploaded image files (JPG, PNG) before passing them to the AI. |
| uuid | Generates unique random IDs (like WX3K9A) for each pet profile, so records don't get confused. |
| json | Reads and writes the app's database files. All pet profiles and the knowledge base are stored as JSON files. |
| os | Interacts with the file system — checks if files exist, creates them, reads file paths. |
| datetime / timedelta | Handles dates and times — recording when profiles were created, calculating days until vet appointments, etc. |
| FPDF (fpdf2) | PDF generation library. Creates the downloadable care guide PDFs with formatted text, sections, and layout. |
| re | Regular expressions — used for text pattern matching and cleaning. |
| urllib.parse | Encodes search terms into URL-safe format (e.g. 'dog food' becomes 'dog+food') for the Azerbaijani store links. |

### Key talking point for pitches

We use transfer learning with MobileNetV2 — a model Google pre-trained on millions of images. This means we get professional-grade AI accuracy without needing to collect training data

or spend months training a model. We simply plug in an existing model and use it for inference. This dramatically reduced development time and cost.

# 3. Azerbaijan E-Commerce Integration

Lines ~30–65 define the local marketplace connections. This is one of Woofer's key differentiators — it's not a generic global app, it's built specifically for Azerbaijan.

## 3.1 AZERBAIJAN_PET_STORES dictionary

This is a Python dictionary (key-value store) that holds the base search URL for three local platforms:

| Platform | Integration details |
|---|---|
| Biopet.az | Azerbaijan's largest online pet shop. We append the product name to their search URL so users land directly on relevant results. |
| Wolt Azerbaijan | Food/delivery platform. Used for quick pet supply delivery in Baku. |
| Tap.az | Local classifieds marketplace. Useful for finding second-hand pet items and local sellers. |

## 3.2 get_azerbaijan_search_links() function

This function takes a product name (e.g. 'salmon-based kibble'), URL-encodes it using urllib.parse.quote(), then appends it to each store's base URL. This generates ready-to-click shopping links without any API integration — simple but effective.

**□ Why this matters for pitches**

Localization is a major competitive advantage. Most pet care apps are built for Western markets and don't integrate Azerbaijani platforms. By connecting directly to Biopet.az, Wolt, and Tap.az, Woofer creates an affiliate revenue stream while solving a real local need.
This is the ethical business model: commissions on supplies, never on animal sales.

# 4. The Knowledge Base — RAG System

Lines ~70–280 contain Woofer's knowledge base and the RAG (Retrieval-Augmented Generation) system. This is one of the most technically impressive parts of the project.

## 4.1 What is RAG?

RAG stands for Retrieval-Augmented Generation. Instead of asking a generic AI to guess care information, we pre-store expert knowledge in a structured database, then retrieve the relevant section when needed. This gives more accurate, reliable, and breed-specific answers than a general chatbot.

### RAG explained simply (use this in pitches)

Imagine a vet with a filing cabinet full of breed-specific care cards.
When you bring in a Golden Retriever, they pull out the Golden Retriever card — not a generic dog card.
That's exactly what our RAG system does. The AI identifies the breed, then we retrieve the specific care card for that breed from our knowledge base.
This is more reliable than asking a chatbot to generate information on the fly.

## 4.2 initialize_knowledge_base() function

This function creates a large Python dictionary with detailed care data for 6 breeds: Siberian Husky, Golden Retriever, German Shepherd, French Bulldog, Labrador Retriever, and a general_dog fallback. Each breed entry contains:

- nutrition — diet type, calories, feeding schedule, recommended/avoided foods
- grooming — brushing frequency, bathing schedule, special notes
- exercise — daily requirements, recommended activities, mental stimulation
- health — common issues, lifespan, vet checkup schedule
- supplies — categorized shopping list with specific product recommendations
- cost_estimate_monthly — estimated care cost in AZN
- good_for & challenges — who the breed suits and what's difficult

The knowledge base is saved to a JSON file (dog_care_knowledge.json) for persistence. On subsequent loads, it reads from the file instead of recreating it.

## 4.3 get_breed_info() function — the RAG retrieval step

After the AI identifies a breed, this function finds the matching entry in the knowledge base. It uses two levels of matching:

- Direct match — checks if any known breed alias appears in the detected label
- Partial match — checks for substring overlap between detected label and breed names
- Fallback — if no match found, returns the general_dog entry so the app always has something useful to show

# 5. The Database — JSON File Storage

Lines ~285–340 handle data persistence. The app stores all pet profiles in a local JSON file called woofer_care_registry.json.

## 5.1 Why JSON instead of a real database?

For a prototype and MVP, JSON file storage is a practical choice. It requires zero setup, works everywhere Python runs, and is human-readable. In production, this would be replaced with a proper database like PostgreSQL or Firebase.

| Function | What it does |
|---|---|
| save_to_db(data) | Appends a new pet profile to the JSON file. Automatically adds a creation timestamp and a unique 8-character profile ID (e.g. WX3K9A). |
| load_all_pets() | Reads and returns all stored pet profiles. Used by the Care Plan and Health Tracker tabs to populate dropdowns. |
| update_pet_profile(id, updates) | Finds a specific profile by ID and updates it with new data. Used for health record updates. |

### Note for technical questions

If asked 'Why not use a real database?' — the honest answer is that this is an MVP (Minimum Viable Product).

JSON storage proves the concept. The architecture is designed so that replacing the JSON functions with database calls (e.g. SQLAlchemy + PostgreSQL) is straightforward — the rest of the code doesn't change.

# 6. PDF Generation — create_care_guide_pdf()

Lines ~343–540 create the downloadable PDF care guide using the FPDF2 library. When a user clicks 'Download Full Care Guide', this function runs and produces a formatted PDF on the fly.

## 6.1 How it works

- Creates a blank PDF page using FPDF()

- Sets fonts, colors, and text sizes to create a professional layout
- Adds sections for Nutrition, Exercise, Health, and Supplies
- Fills each section with data pulled from the knowledge base
- Returns the PDF as bytes which Streamlit serves as a file download

### 6.2 The fixed line (important to know)

The original code had a compatibility bug on this line:

```
return pdf.output(dest='S').encode('latin-1')   # OLD — broken in fpdf2
```

The fix was:

```
return bytes(pdf.output())   # NEW — fpdf2 returns bytes directly
```

This is because fpdf2 (the modern version) changed its API — output() now returns bytes directly instead of a string that needed encoding. This is an example of a breaking API change between library versions, a common real-world challenge.

## 7. The AI Model — Computer Vision

Lines ~545–630 contain the core AI functionality — the breed detection engine.

### 7.1 load_woofer_model() — Loading MobileNetV2

```
@st.cache_resource
```

The @st.cache_resource decorator is critical for performance. It means the model is loaded into memory once when the app starts, not every time a user uploads a photo. Loading a neural network takes ~10 seconds; caching means users get instant responses after the first load.

MobileNetV2 is loaded with weights='imagenet', meaning it uses Google's pre-trained weights from training on 1.4 million images across 1,000 categories including 120 dog breeds.

### 7.2 predict_breed() — The inference pipeline

This function runs the 5-step AI inference pipeline:

1. Resize image to 224×224 pixels — MobileNetV2's required input size
2. Convert image to numpy array — transforms pixels into numbers the model can read
3. Add batch dimension — model expects input shape (1, 224, 224, 3) not (224, 224, 3)
4. preprocess_input() — normalizes pixel values from 0-255 to -1 to 1, which is what MobileNetV2 was trained with
5. model.predict() → decode_predictions() — runs the forward pass through 154 neural network layers and returns the top 5 most likely labels with confidence scores

### 7.3 Breed mapping

MobileNetV2 uses ImageNet labels like 'siberian_husky' or 'golden_retriever'. The dog_breeds dictionary in the function maps these ImageNet labels to our knowledge base keys. The function loops through the top 5 predictions and returns the first one that matches a known dog breed.

**□ How to explain the AI to non-technical people**

Imagine showing a photo to someone who has seen 1.4 million images.
They've seen so many dogs, they can recognize a Husky vs a Malamute just from the photo.
MobileNetV2 works the same way — it learned visual patterns from millions of images.
We show it a new photo, it compares it to everything it learned, and tells us
the most likely breed with a confidence percentage like 87.3%.

## 8. The User Interface — Streamlit Components

Lines ~635–1084 build the entire visual interface. Streamlit works by running the Python script top-to-bottom every time a user interacts with anything. Each UI function renders a specific part of the page.

| Function | What it renders |
|---|---|
| render_header() | The top of the page — app logo, title, and tagline using a 2-column layout. |
| render_sidebar() | The left panel — mission statement, how-it-works summary, and a live counter of how many pets have been analyzed. |
| render_breed_analysis() | Tab 1 — The main feature. File uploader, pet info form, AI analysis trigger, results display, PDF download button. |
| render_care_recommendations() | Tab 2 — Detailed care plan with 5 sub-tabs (Nutrition, Exercise, Grooming, Health, Supplies) and Azerbaijan shopping links. |
| render_health_tracker() | Tab 3 — Overview of all pet profiles, exercise tracker with progress bar, vet appointment reminders. |
| render_adoption_support() | Tab 4 — Ethical stance page, pre-adoption checklist, local Azerbaijani shelter resources. |
| main() | The entry point — initializes the knowledge base, calls render_header() and render_sidebar(), then creates the 4 tabs and calls each render function inside its tab. |

### 8.1 Session State — st.session_state

Streamlit reruns the entire script on every interaction. To 'remember' things between reruns (like the current pet analysis result), the app uses st.session_state — a dictionary that persists across reruns. Key entries:

- st.session_state['current_pet'] — stores the most recently analyzed pet's data
- st.session_state['analysis_complete'] — flag to show/hide the results panel
- st.session_state['selected_pet_id'] — tracks which pet is selected in the health tracker

### 8.2 Forms and the submit pattern

The breed analysis uses st.form() with a submit button. This is important because it groups all the inputs (nickname, age, weight) together so the AI analysis only runs when the user explicitly clicks 'Analyze' — not on every keystroke. Inside the form, st.form_submit_button() triggers the analysis, followed by st.rerun() to refresh the display with the new results.

## 9. Deployment — How It's Live on the Internet

The app is deployed on Streamlit Community Cloud, which is free for public repositories. Here's the full deployment stack:

| Layer | Technology & details |
|---|---|
| Source code | GitHub — github.com/tmammadov17503/Woofer_project |
| App hosting | Streamlit Community Cloud — wooferproject.streamlit.app |
| Landing website | GitHub Pages — tmammadov17503.github.io/Woofer_project |
| Python version | Python 3.13 (Streamlit Cloud default) |
| AI framework | tensorflow-cpu >= 2.20 (CPU-only, sufficient for inference) |
| Dependency file | requirements.txt in repo root — pip installs from this on deploy |
| Auto-deploy | Every git push to main branch triggers automatic redeployment |

### 9.1 The requirements.txt — why it matters

Without requirements.txt, Streamlit Cloud would not know which libraries to install. The file specifies:

```
streamlit>=1.28.0
tensorflow-cpu>=2.20.0
numpy>=1.26.0
Pillow>=9.0.0
fpdf2>=2.7.0
```

```
requests>=2.28.0
```

tensorflow-cpu is used instead of the full tensorflow package because Streamlit Cloud runs on CPU servers (no GPU). The CPU version is lighter and works perfectly for inference (running predictions) — we don't need GPU power because we're not training the model.

# 10. Common Questions You May Be Asked

### Q: What AI model do you use and why?

We use MobileNetV2, a convolutional neural network developed by Google. We chose it because it's lightweight (fast inference on CPU), highly accurate on image classification tasks, pre-trained on ImageNet which includes 120+ dog breeds, and open-source. Using a pre-trained model means we got professional-grade AI without months of training time or large datasets.

### Q: What is RAG and why did you use it?

RAG stands for Retrieval-Augmented Generation. Instead of relying on a generative AI to make up care information (which can hallucinate or be wrong), we store verified, expert-curated breed care data in a structured knowledge base, then retrieve the exact entry when a breed is detected. This gives us 100% accurate, predictable, auditable recommendations — crucial in a health-related application.

### Q: How accurate is the breed detection?

MobileNetV2 achieves ~71% top-1 accuracy on ImageNet. For dog breeds, accuracy is typically higher with clear, well-lit photos. The app shows a confidence score for transparency. For breeds not in our knowledge base, it falls back to general dog care information. This is honest about limitations while still being useful.

### Q: How does the app make money?

The business model is freemium + ethical affiliate marketing. Basic features are free. Premium subscriptions (4.99 and 9.99 AZN/month) unlock advanced features. Affiliate commissions come from pet supply purchases through Biopet.az, Wolt, and Tap.az. Critically, we never earn from pet sales or breeding — this is enforced by design, not just policy.

### Q: What would you improve with more resources?

Three main areas: First, expand the knowledge base from 6 breeds to 120+ breeds with a more robust RAG system or integration with a veterinary database. Second, replace JSON file storage with a proper database (PostgreSQL) and add user authentication. Third, fine-tune a custom computer vision model specifically trained on dog breeds rather than relying on general ImageNet weights — this would improve breed detection accuracy significantly.

### Q: Why Streamlit instead of a traditional web framework?

Streamlit lets us build and deploy a full interactive web application in pure Python, without writing any HTML, CSS, or JavaScript for the app itself. For an MVP/prototype, this dramatically accelerates development — the entire app is ~1,100 lines of Python. In production, we'd likely migrate to a React frontend with a FastAPI backend for more control and performance.

## 11. Architecture Summary

### □ Full System Architecture

```
USER → uploads photo via browser
    ↓
STREAMLIT (frontend + backend) → renders UI, handles file upload
    ↓
PILLOW → opens and preprocesses the image
    ↓
MOBILENETV2 (TensorFlow) → runs AI inference, returns breed + confidence
    ↓
RAG SYSTEM → matches breed to knowledge base entry
    ↓
KNOWLEDGE BASE (JSON) → returns full care data for that breed
    ↓
UI RENDER → displays results, care plan, shopping links
    ↓
FPDF2 → generates downloadable PDF on demand
    ↓
DATABASE (JSON) → saves pet profile for future reference
```

*Woofer Care AI — Promoting Responsible Pet Ownership Through Technology*

© 2025 Woofer Project · Azerbaijan □□ · wooferproject.streamlit.app